Résolution informatique du jeu logique forteresse :
stratégie et modélisation

Maxime Saffon n°19565

TABLE DES MATIÈRES

1 Les règles du jeu 3

2 La modélisation informatique 11

3 Objectif du TIPE 14

4 Première approche : un algorithme force brute 15

5 Deuxième approche : le backtracking 16

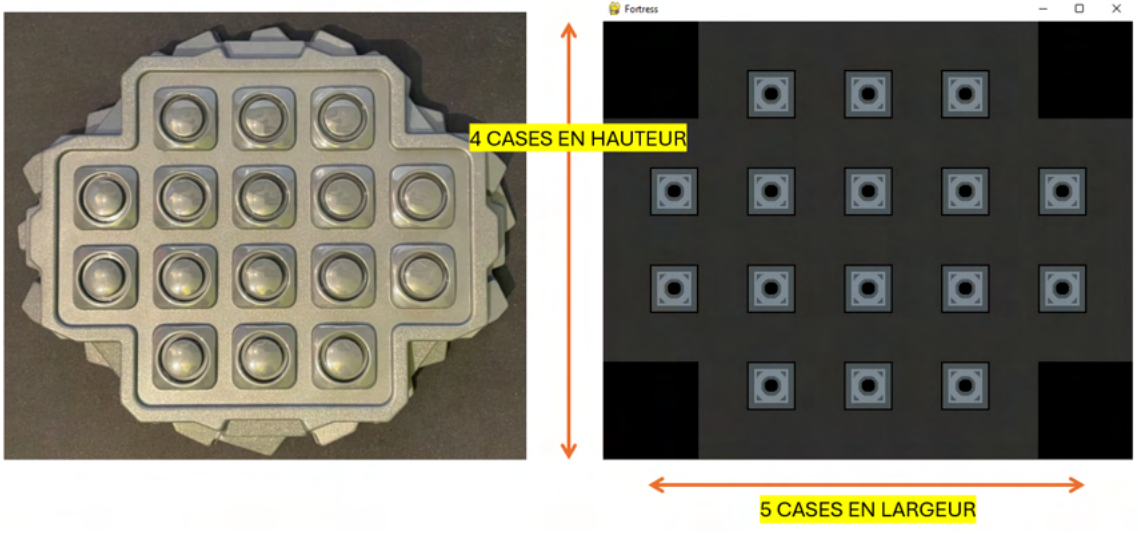
6 Première optimisation : un prétraitement des données 23

7 Deuxième tentative d'optimisation : utilisation de l'algorithme A* 32

8 Annexe 60

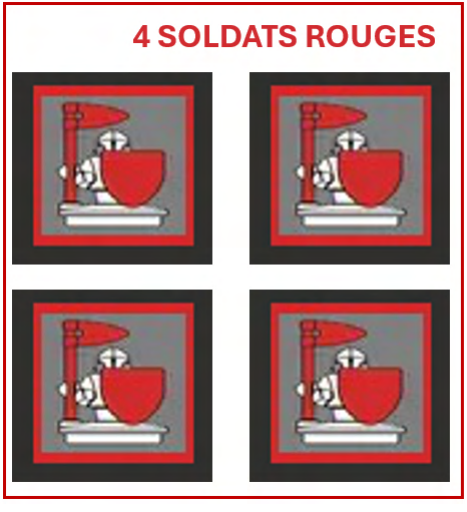
LES RÈGLES DU JEU

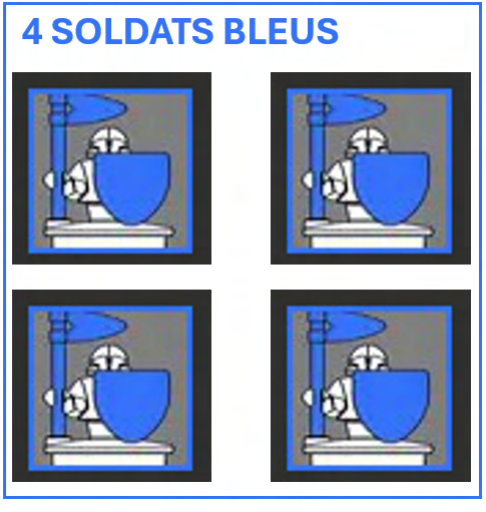
LA GRILLE



LES RÈGLES DU JEU

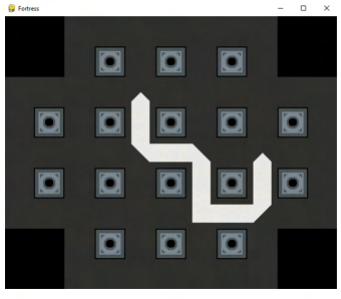
LES SOLDATS

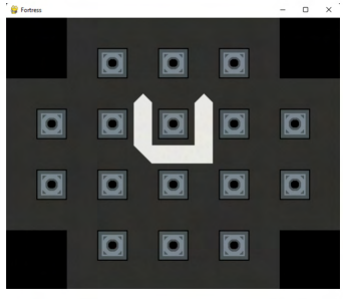


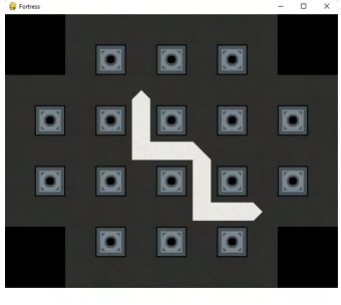


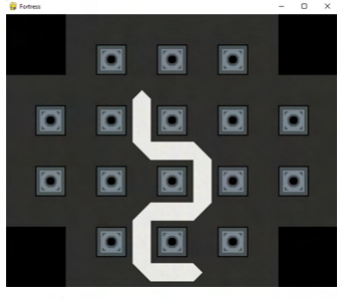
LES RÈGLES DU JEU

LES REMPARTS



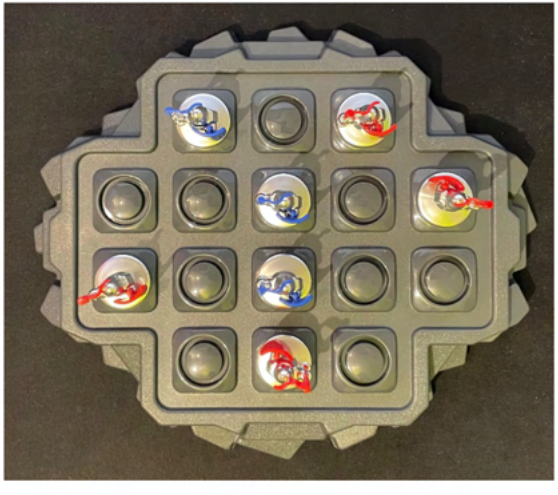


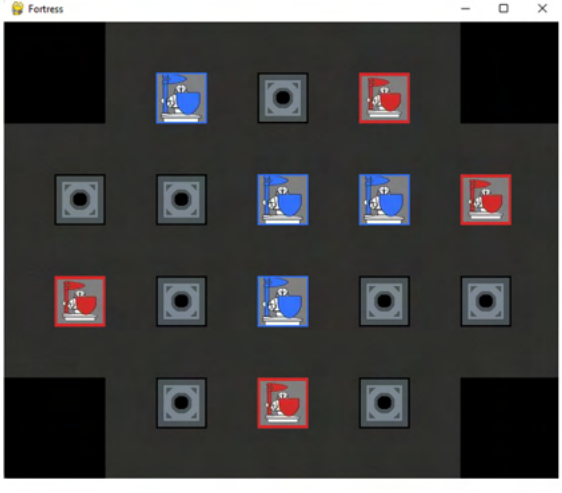




LES RÈGLES DU JEU

LES DÉFIS

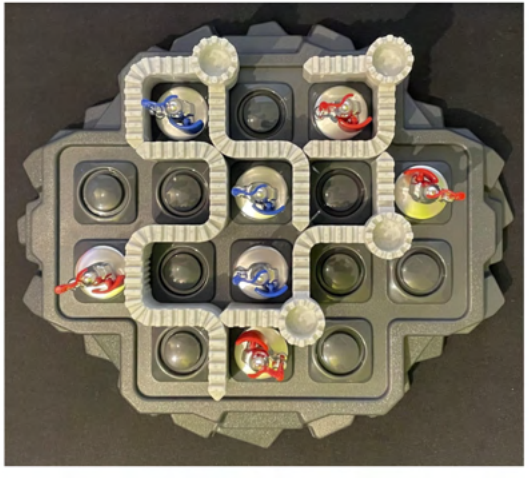


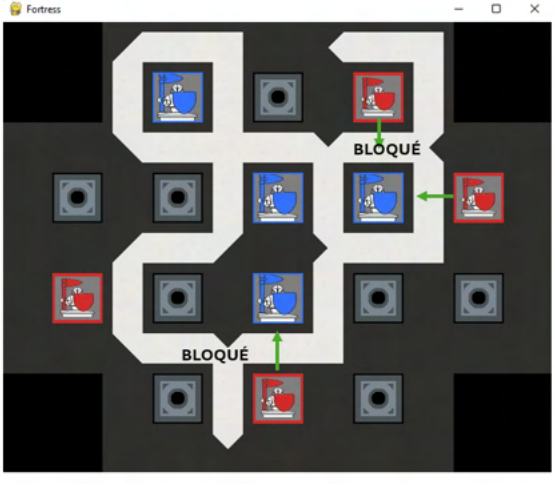


LES RÈGLES DU JEU

OBJECTIF

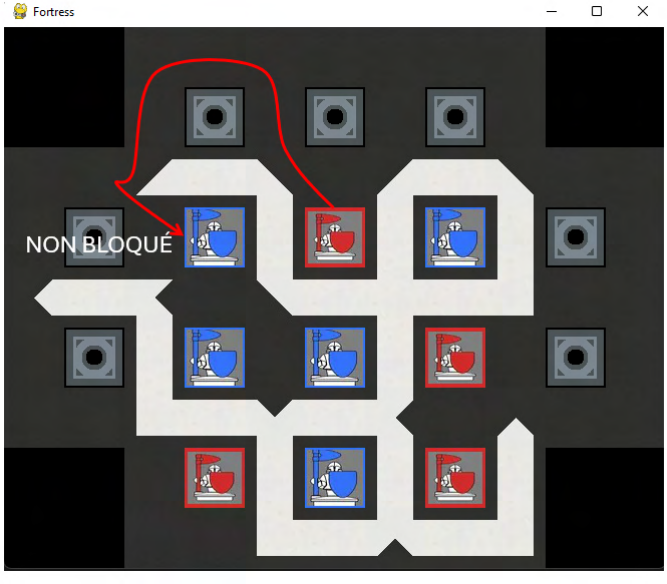
OBJECTIF : CRÉER UN COUNTOUR AFIN QUE LES PIONS ROUGES NE PUISSENT PAS ATTAQUER LES BLEUS





LES RÈGLES DU JEU

OBJECTIF



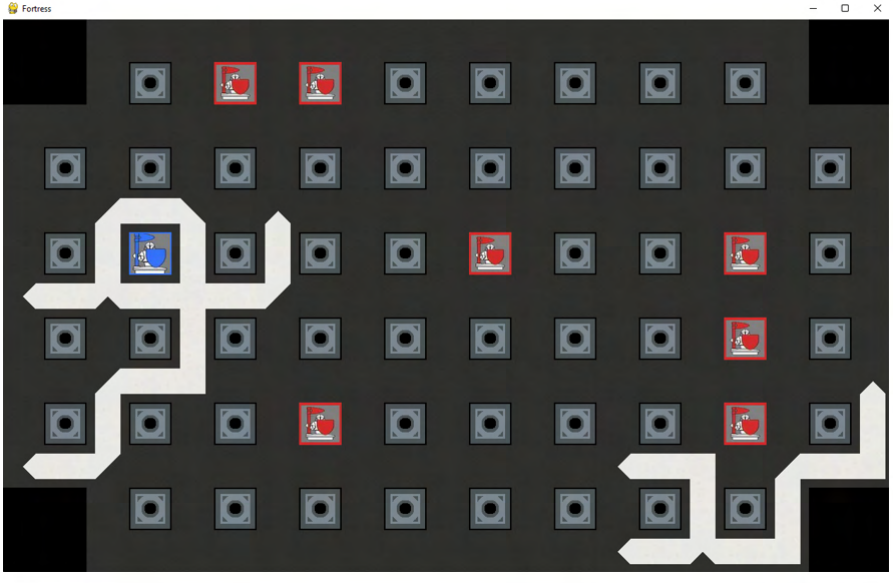
LES RÈGLES DU JEU

UNE GÉNÉRALISATION DU JEU

	LE JEU CLASSIQUE	LE JEU GÉNÉRALISÉ
DIMENSION DE LA GRILLE	4×5	$n \times p$ avec $n,p \in \mathbb{N}$
SOLDATS BLEUS	4	$k \in \mathbb{N}$
SOLDATS ROUGES	4	$k' \in \mathbb{N}$
LES REMPARTS	4	$r \in \mathbb{N}$

LES RÈGLES DU JEU

UN EXEMPLE

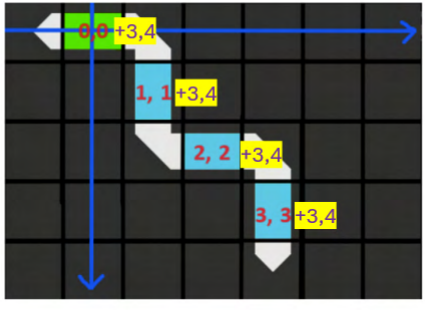


```
val grille : int array array
```

PIÈCES	VALEUR DANS LA GRILLE
-1	CASE INTERDITE
0	SOLDAT NON PLACÉ
1	REMPART NON PLACÉ
2	REMPART PLACÉ
3	SOLDAT BLEU
4	SOLDAT ROUGE

La modélisation informatique

Les remparts




```
val rempart : (int * int) array = [|(0, 0); (1,  1); (2,  2); (3,  3)|]
```

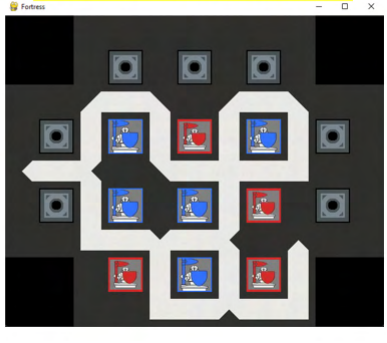
```
val orientation : int = 90
```

```
val position_intial : int * int = (3,4)
```

La modélisation informatique
le module graphique

CSV

```
-1,-1,1,1,1,2,2,1,1,-1,-1
-1,-1,1,3,1,3,2,0,1,-1,-1
1,1,1,2,1,2,1,2,2,1,1
1,0,2,52,1,0,1,0,2,0,1
2,2,1,1,1,2,1,1,1,2,2
2,4,1,51,2,0,2,4,2,0,2
1,2,1,2,1,1,1,2,1,1,1
-1,-1,1,0,1,3,1,3,1,-1,-1
-1,-1,1,1,1,1,1,1,1,-1,-1
```



OBJECTIF DU TIPE

DEUX OBJECTIFS DIFFÉRENTS

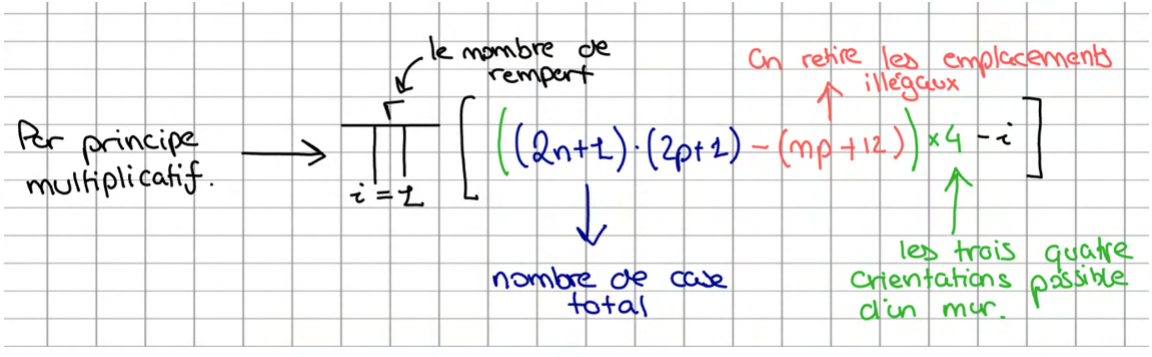
TIPE : LE JEU FORTERESSE

MON OBJECTIF : RÉSOUDRE ALGORITHMIQUEMENT LE JEU FORTERESSE

OBJECTIF DE MON PARTENAIRE : GÉNÉRÉ DES INSTANCES DU JEU DE FORTERESSE

Force brute

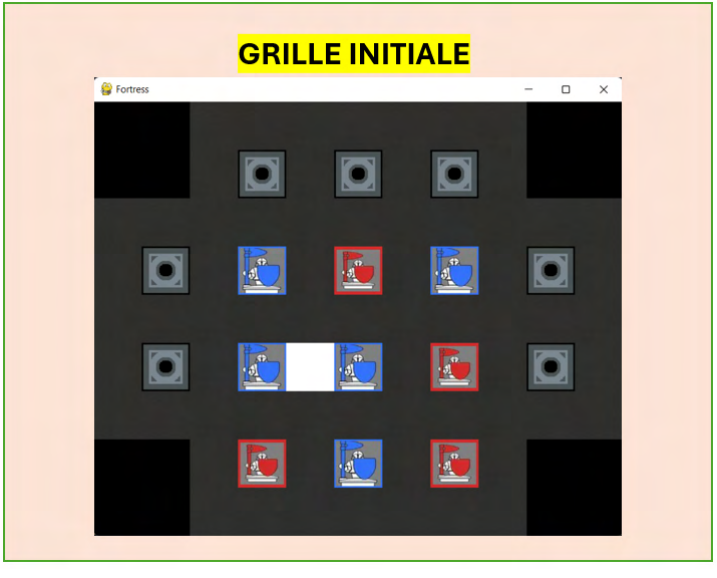
Dénombrement du nombre de possibilités



Donc environ $(12 \times n \times p)^r$ possibilités

Backtracking

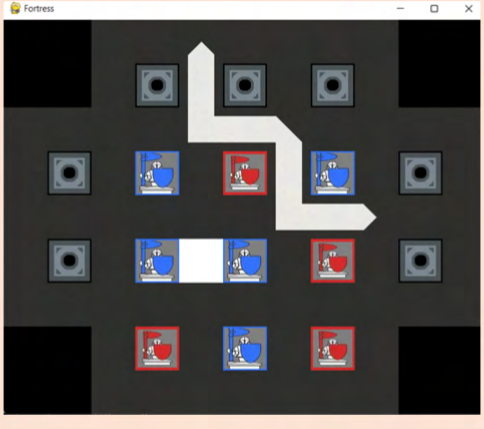
La méthode



Backtracking

La méthode

PLACEMENT DU REMPART 1 AVEC ORIENTATION 1



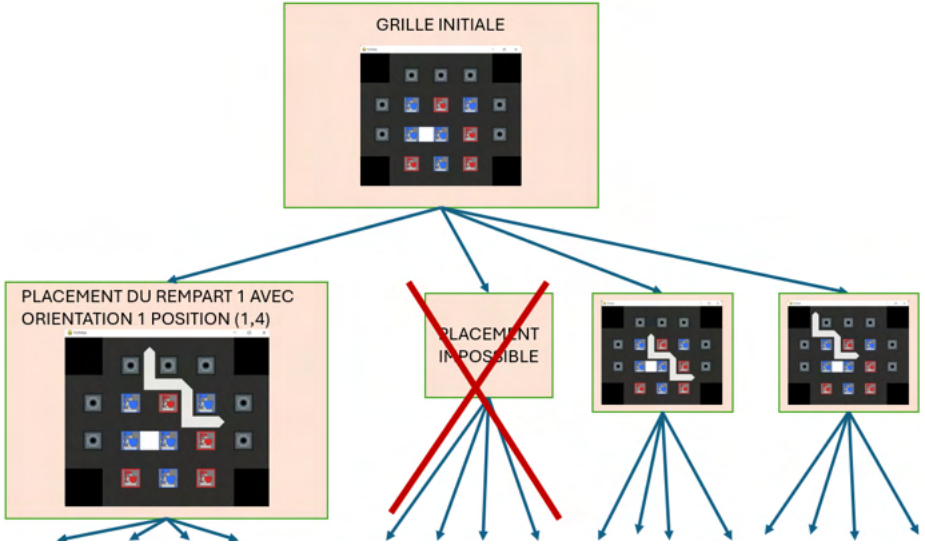
UTILISATION DE

```
let placer_rempart (rempart : (int*int) array) ((x,y) : (int*int)) (f1,f2) (g : int array array) : bool =
```

en O(longueur du rempart)

Backtracking

La méthode



Backtracking

La méthode

UNE GRILLE REMPLIE ÉVENTUELLEMENT SOLUTION



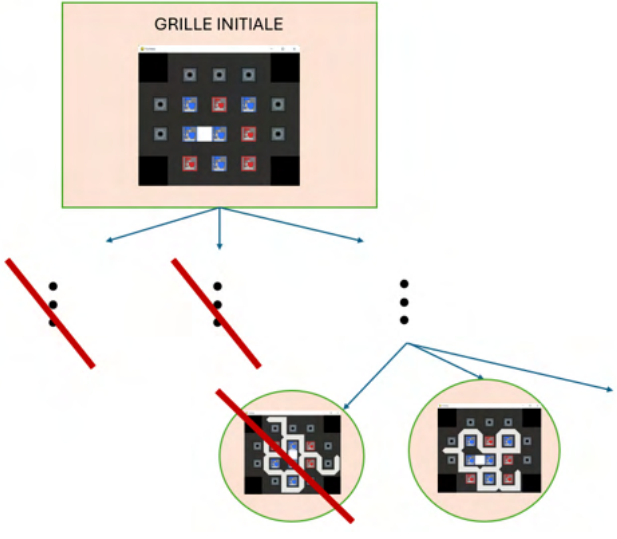
UTILISATION DE

```
let est_solution (g : int array array) : (int*int) list =
```

en O(n×p)

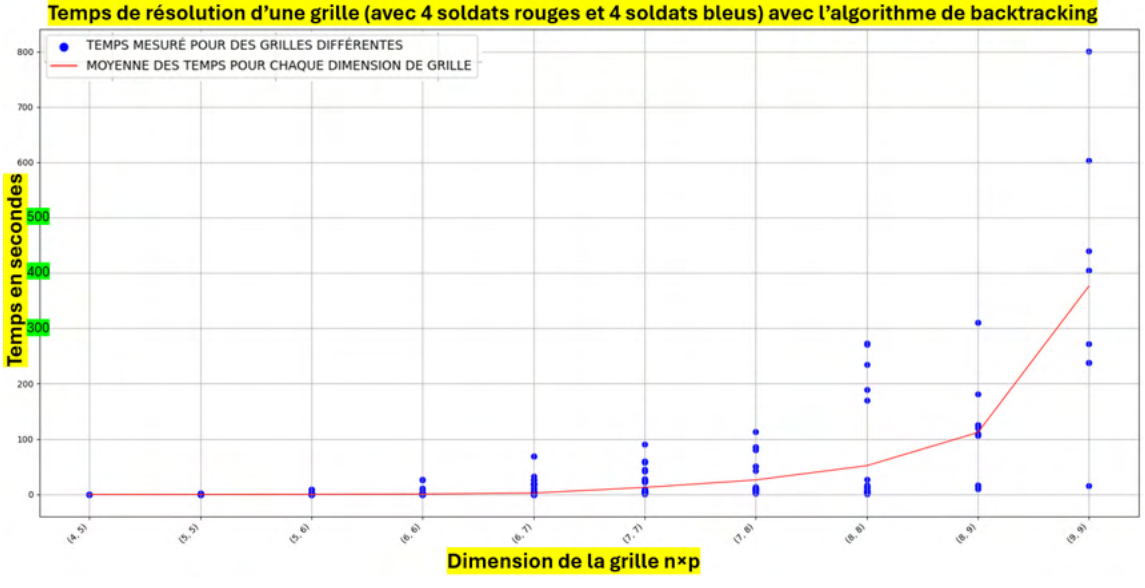
Backtracking

La méthode



BACKTRACKING

Les résultats temporels



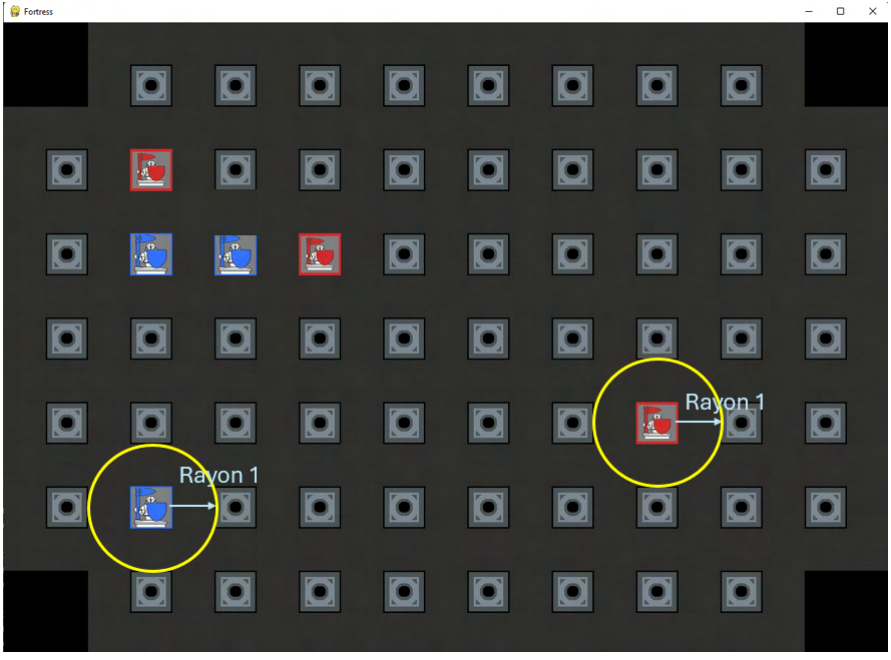
BACKTRACKING

Les résultats temporels



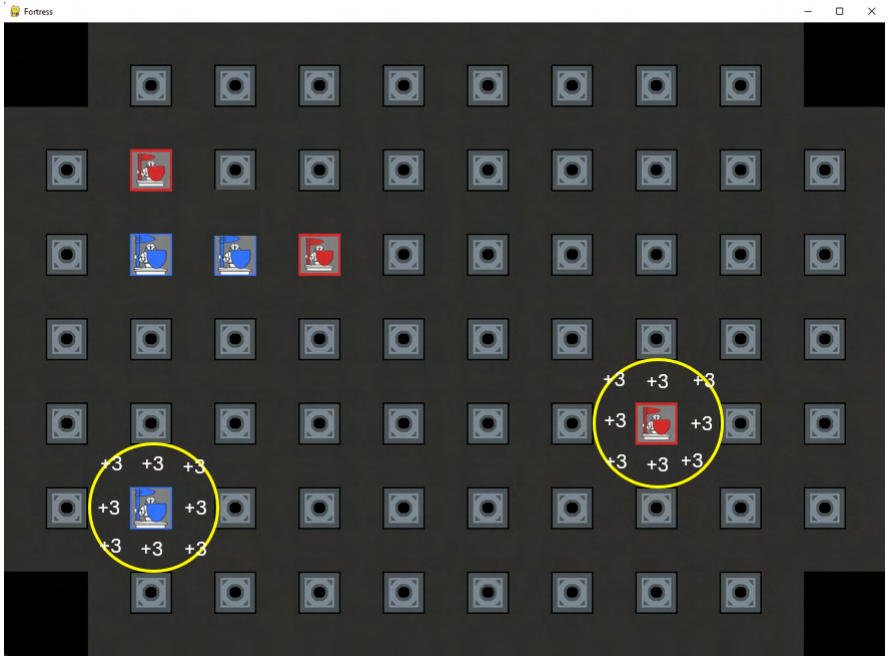
Première optimisation :un prétraitement des données

Explication de la méthode sur un exemple



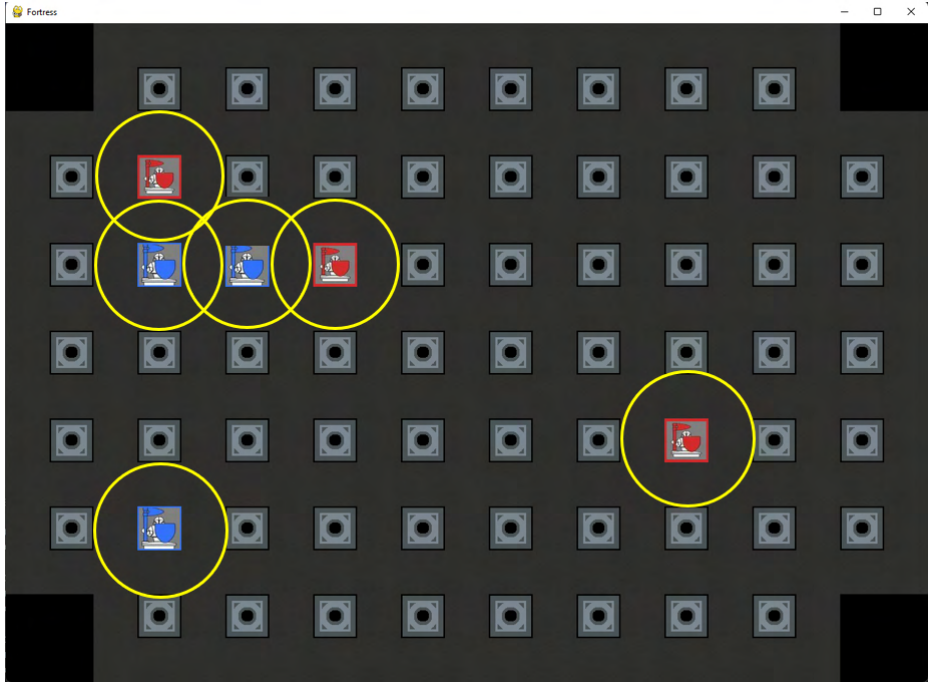
Première optimisation :un prétraitement des données

Explication de la méthode sur un exemple



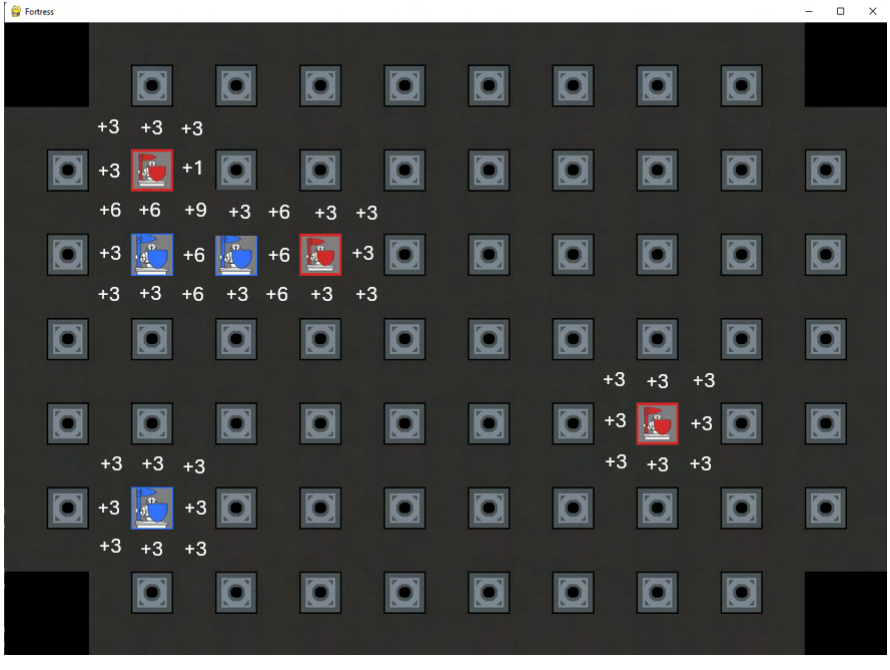
Première optimisation :un prétraitement des données

Explication de la méthode sur un exemple



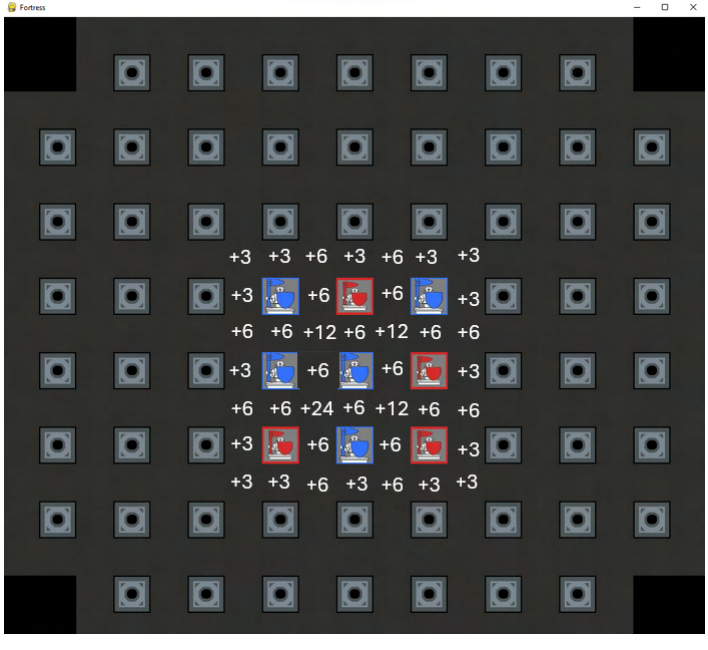
Première optimisation :un prétraitement des données

Explication de la méthode sur un exemple



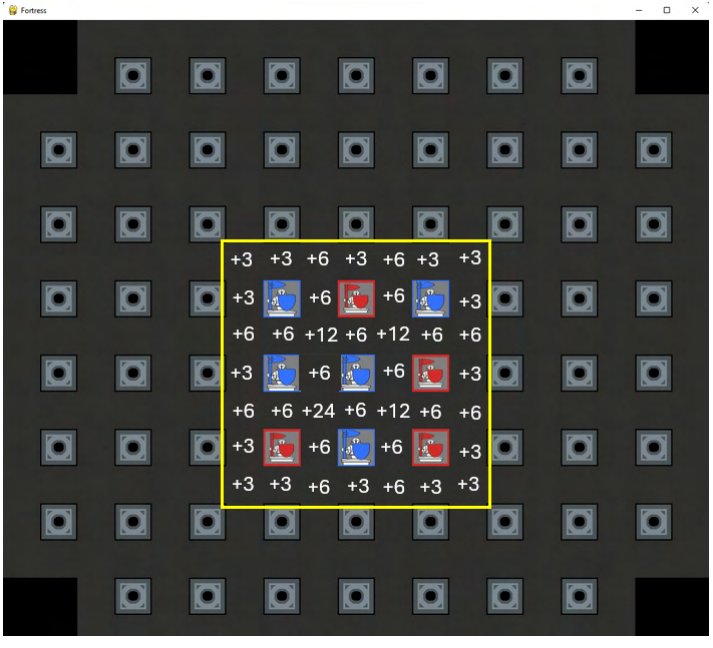
Première optimisation :un prétraitement des données

Explication de la méthode sur un exemple



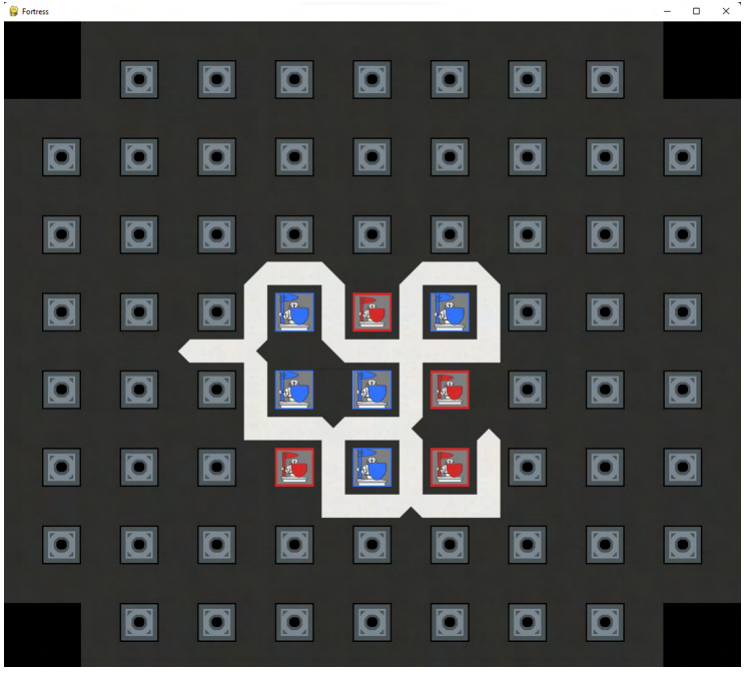
Première optimisation :un prétraitement des données

Un exemple



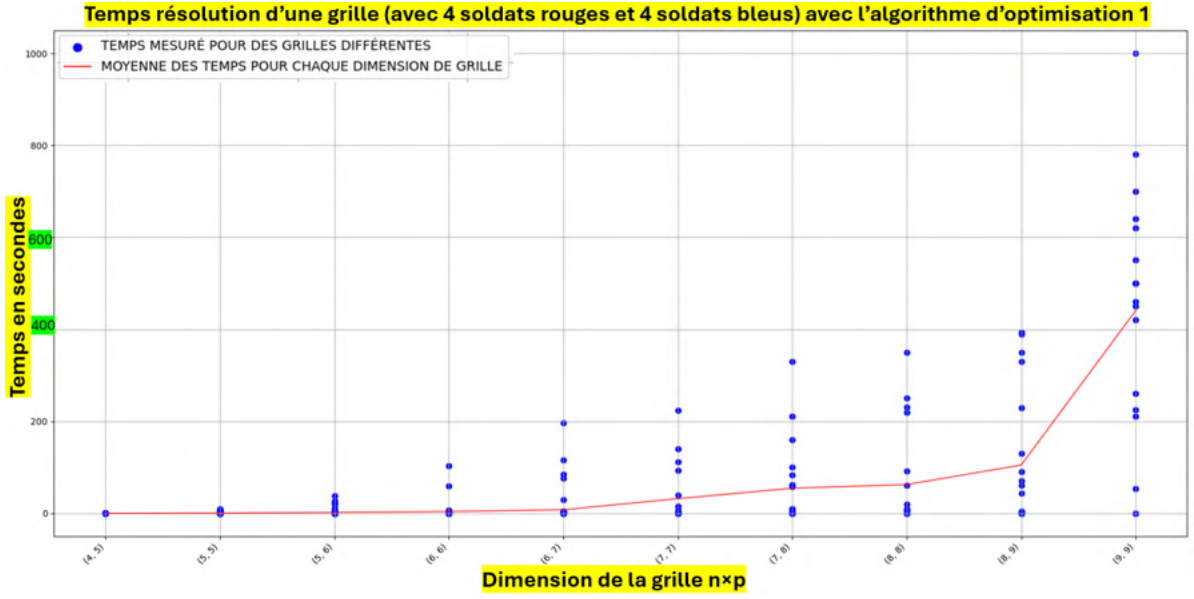
Première optimisation :un prétraitement des données

Un exemple



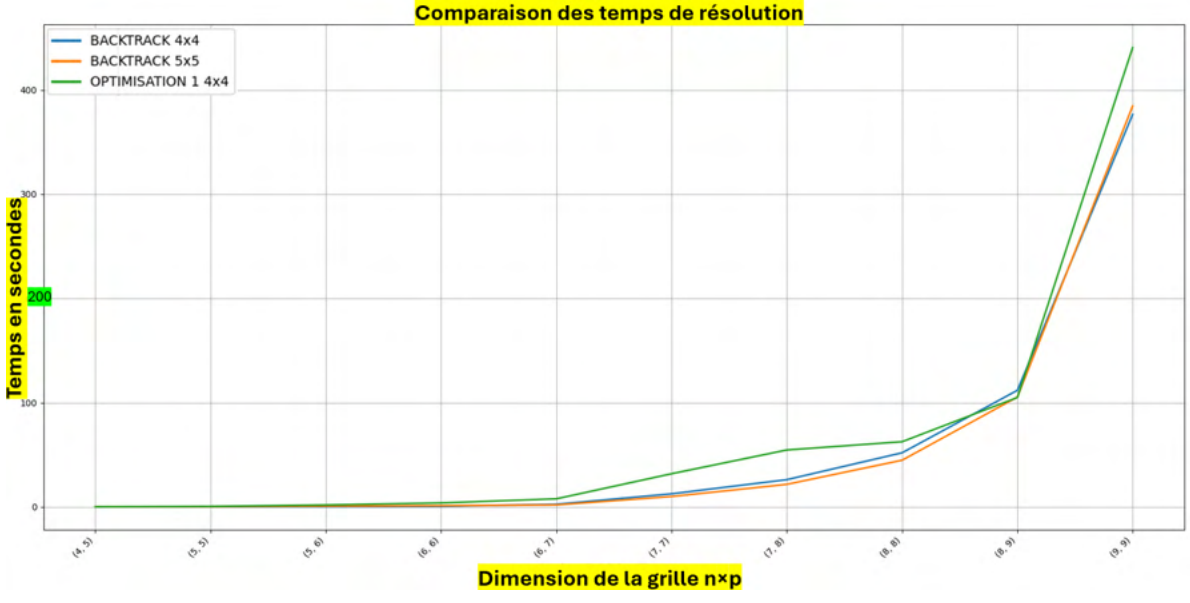
Première optimisation :un prétraitement des données

Résultats temporels



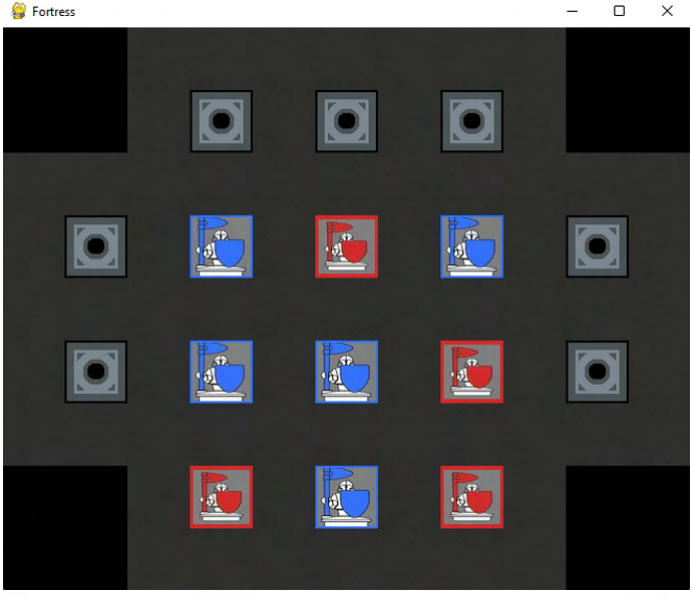
Première optimisation :un prétraitement des données

Comparaison des résultats



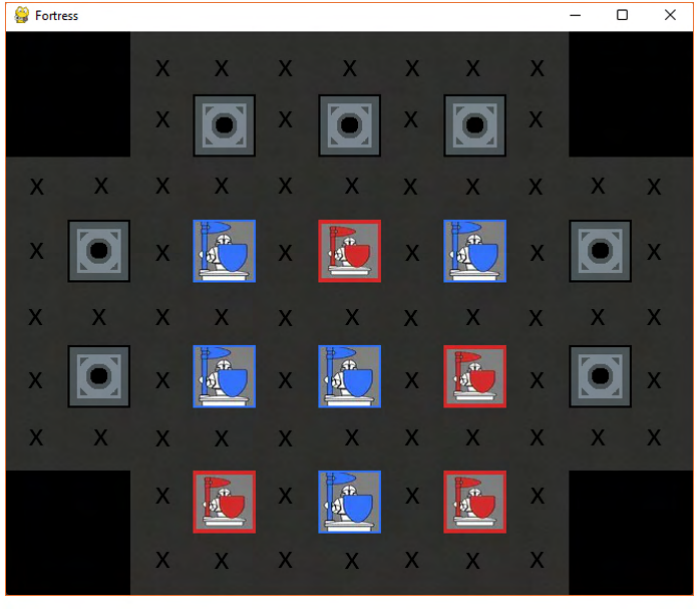
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation



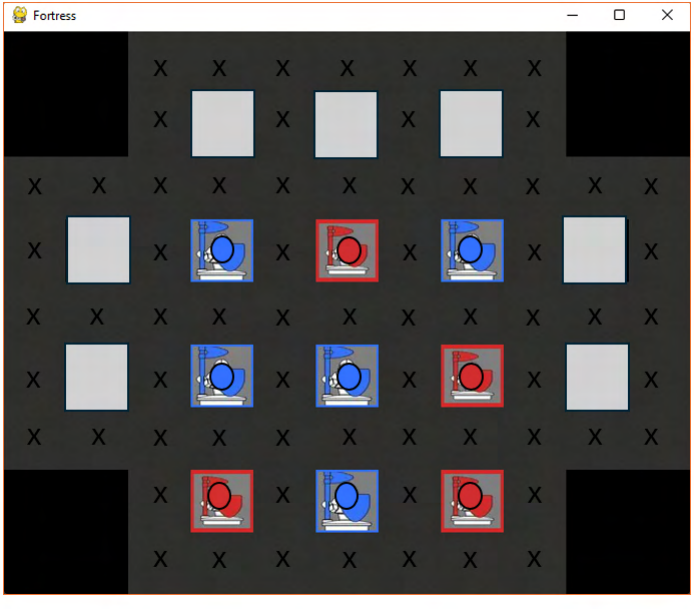
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les sommets "remparts" du graphe



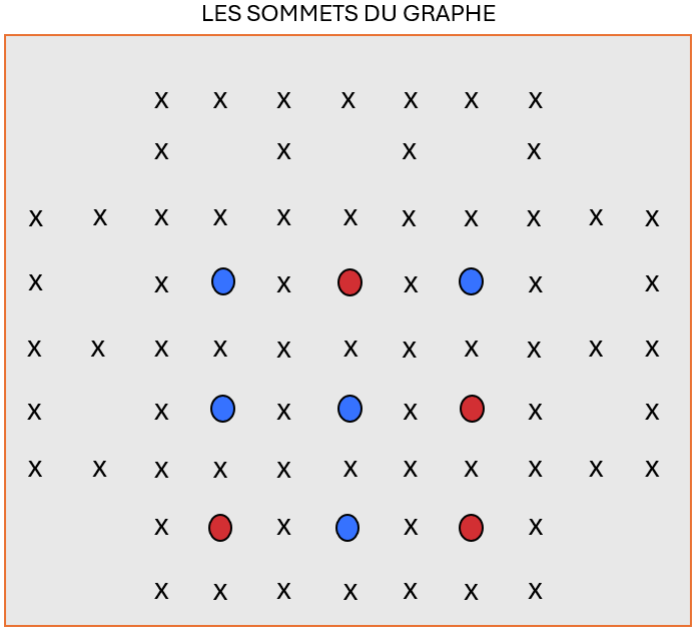
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les sommets "soldats" du graphe



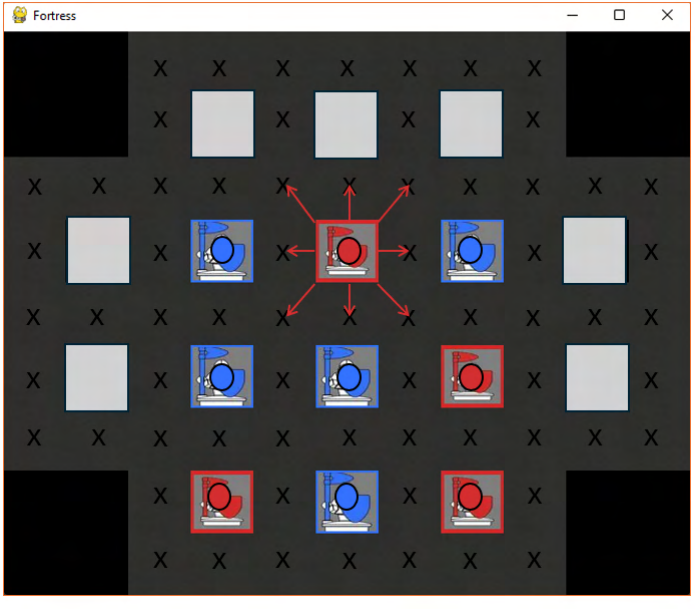
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Un exemple



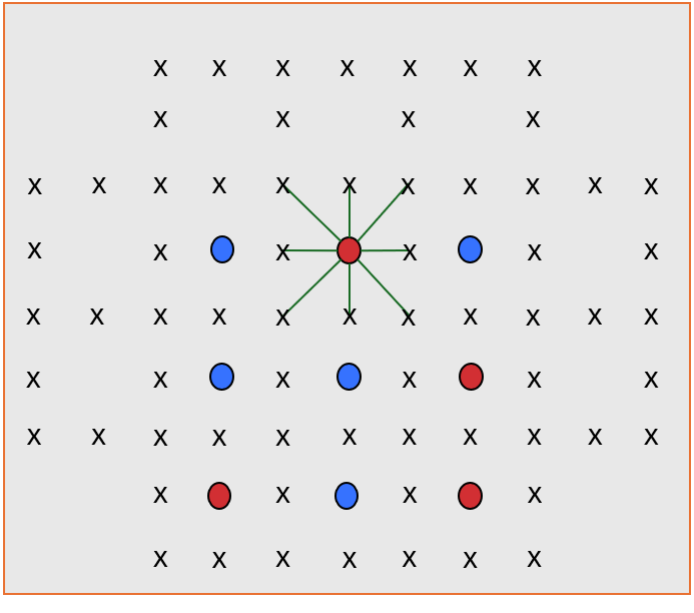
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Déplacement possible d'un sommet dans le graphe



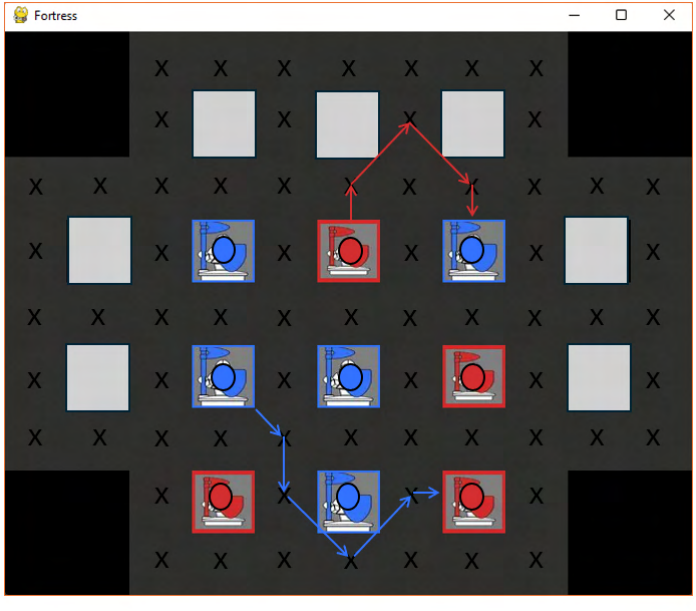
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les arêtes dans le graphe



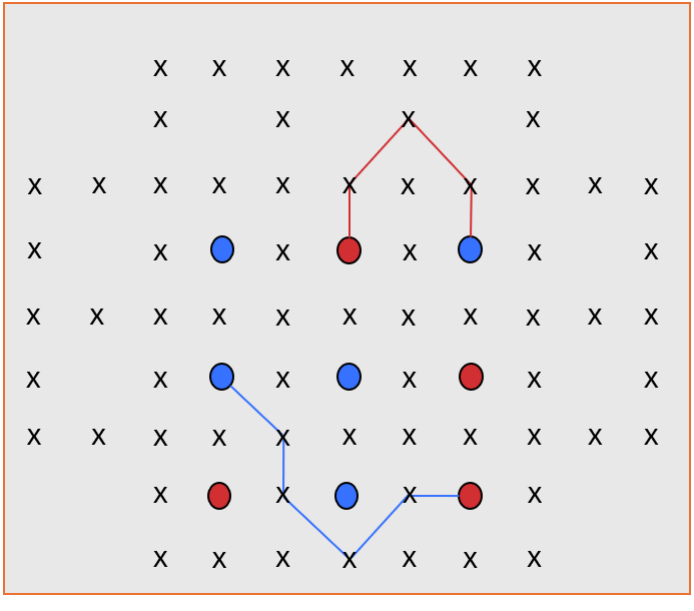
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Déplacement possible d'un sommet dans le graphe



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les arêtes dans le graphe



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

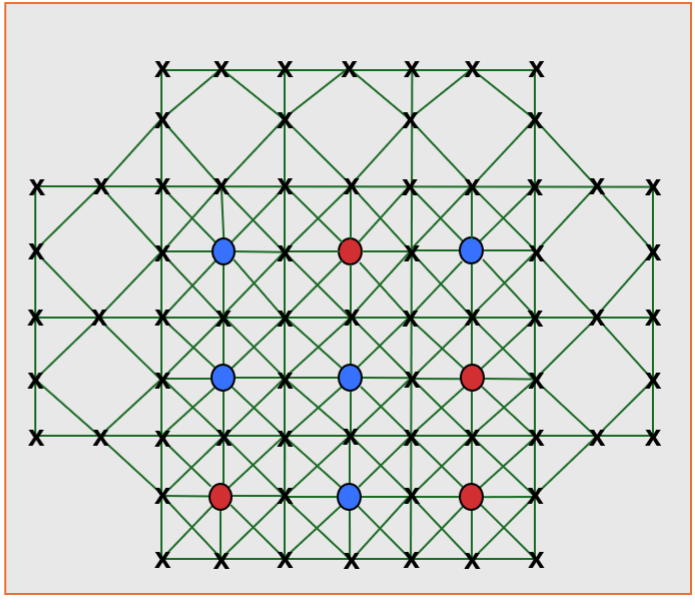
Une nouvelle Modélisation - Un exemple Complet

```
val graphe : (int * int list) array
```

x : (1,voisins)

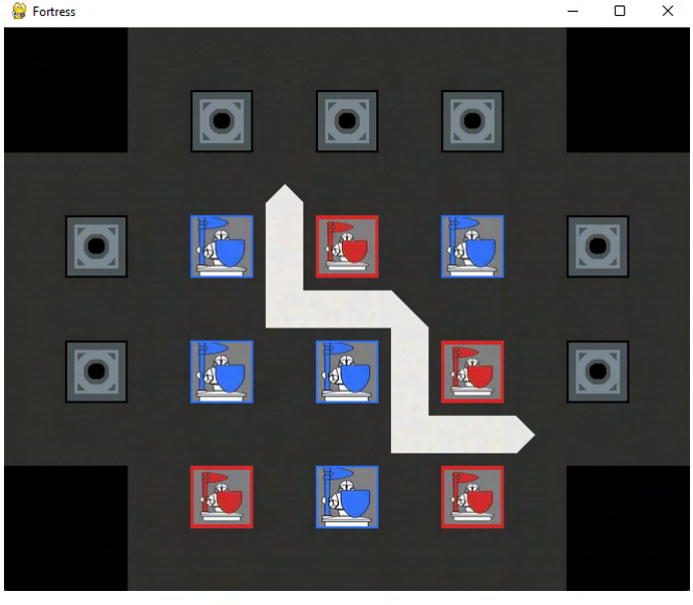
: (4, voisins)

: (3, voisins)



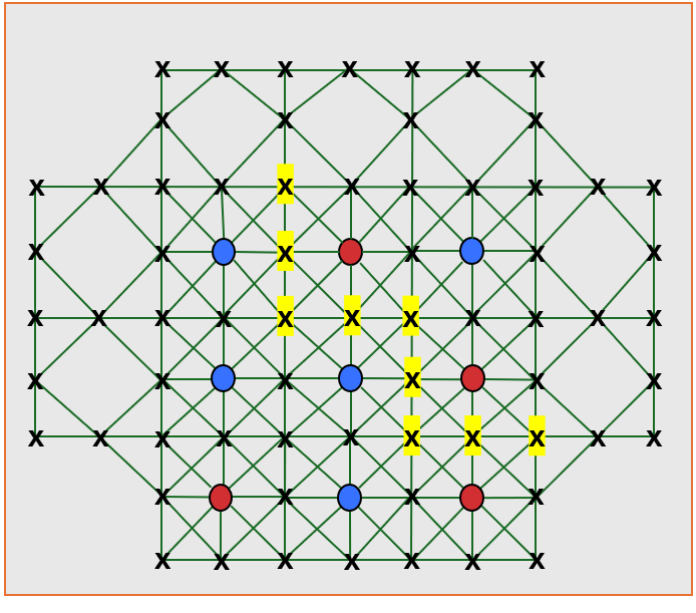
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les remparts



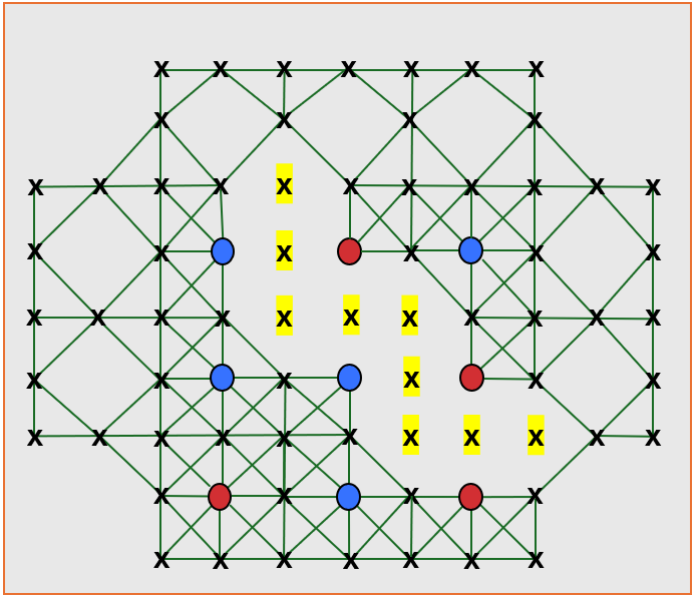
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les remparts dans le graphe



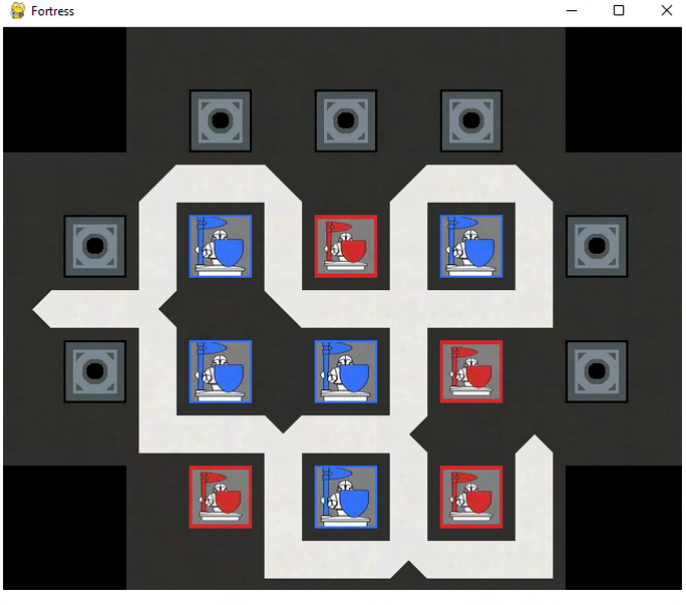
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Les remparts dans le graphe



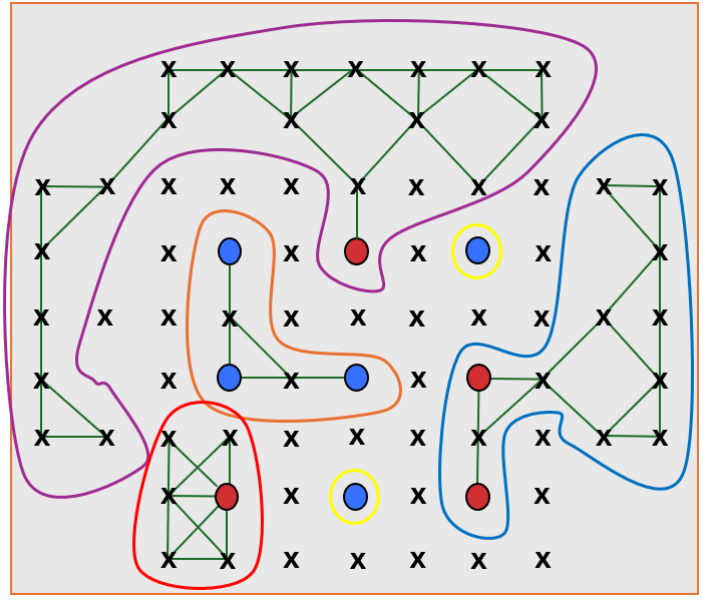
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Un exemple



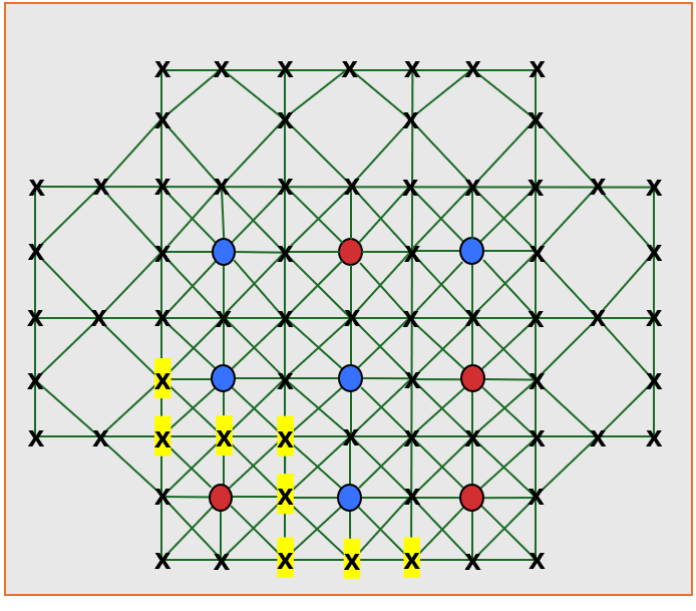
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Une nouvelle Modélisation - Un exemple



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

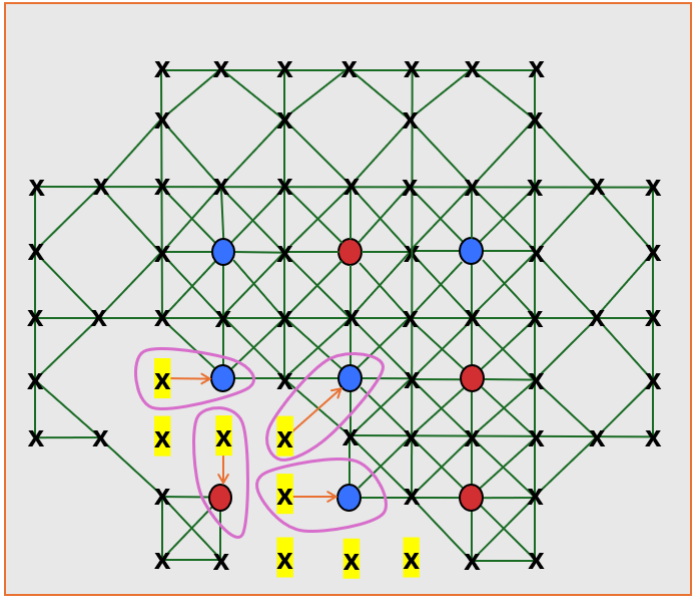
Explication de l'algorithme sur un exemple - Placement d'un mur



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - Repère les sommets "soldats" à distance 1

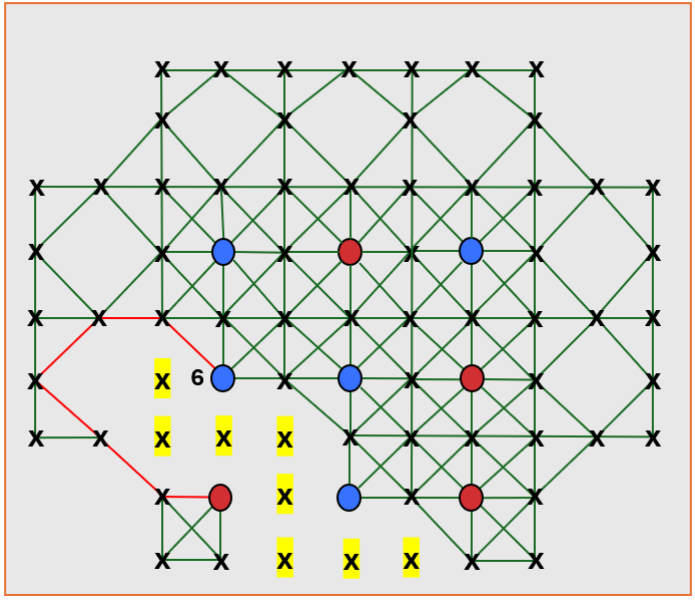
ON CONSIDERE LES SOMMETS À DISTANCE 1DU MUR



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple

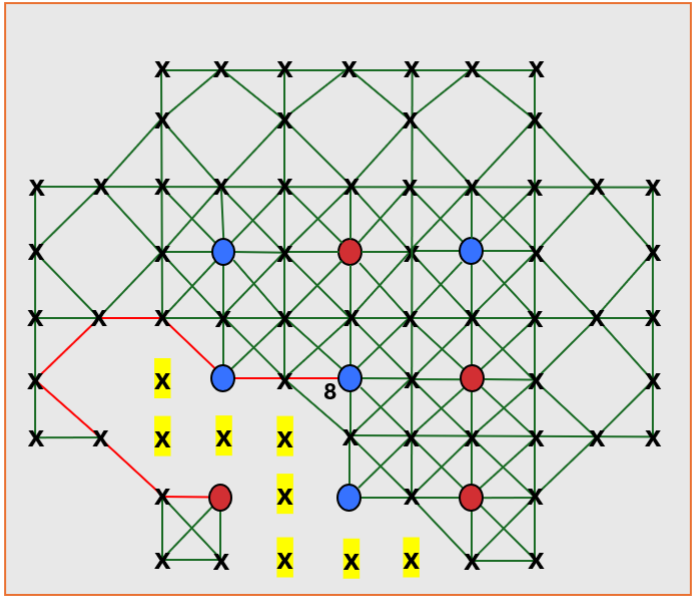
ON CALCUL LE PLUS COURT CHEMIN DE CHAQUE COUPLE DE SOMMET ROUGE ET BLEU



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple

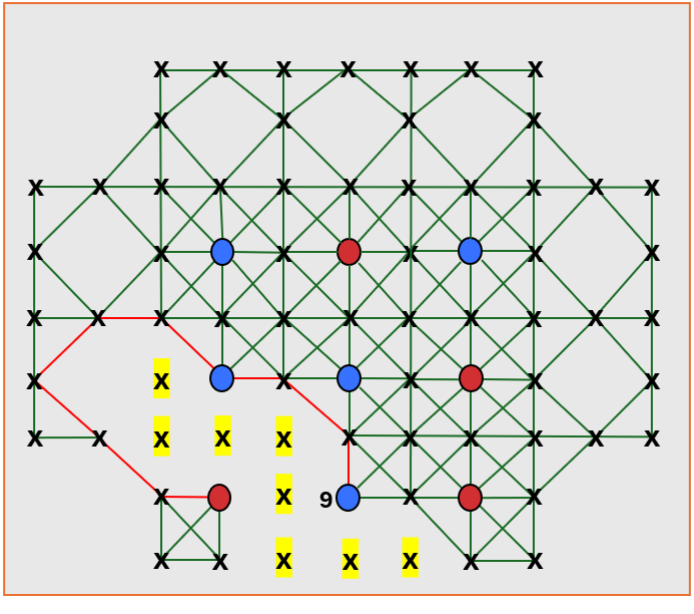
ON CALCUL LE PLUS COURT CHEMIN DE CHAQUE COUPLE DE SOMMET ROUGE ET BLEU



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

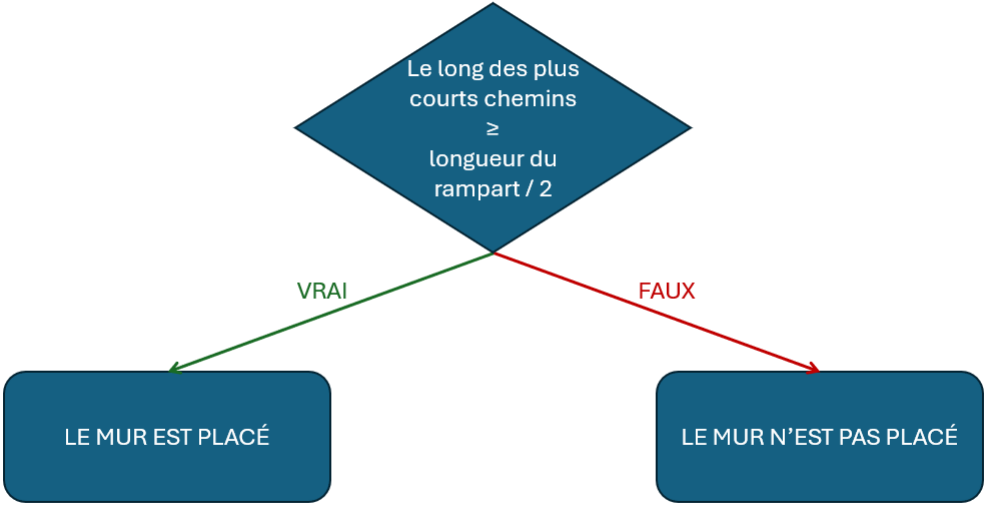
Explication de l'algorithme sur un exemple - exemple

ON CALCUL LE PLUS COURT CHEMIN DE CHAQUE COUPLE DE SOMMET ROUGE ET BLEU



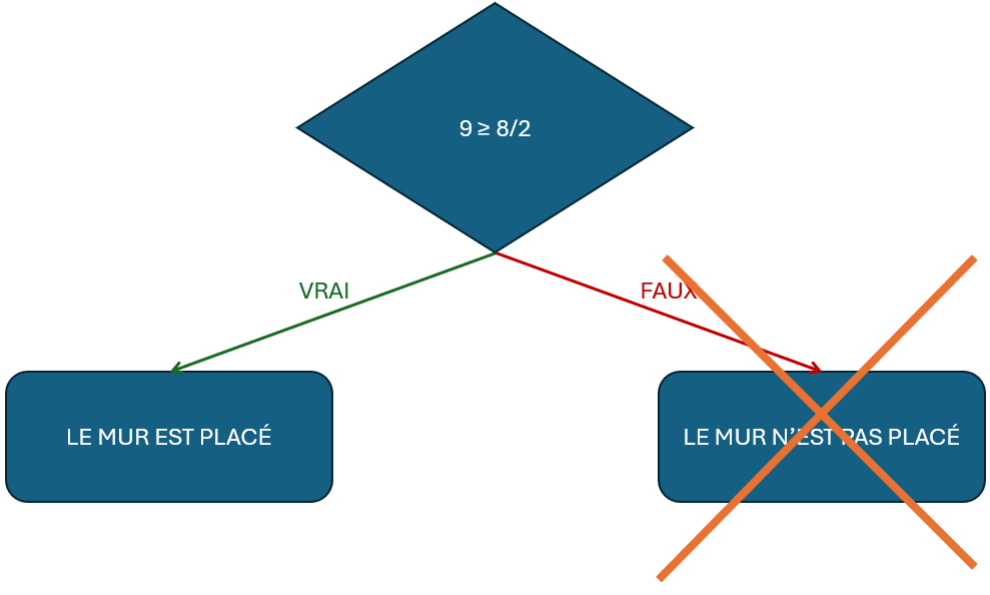
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - Approximation



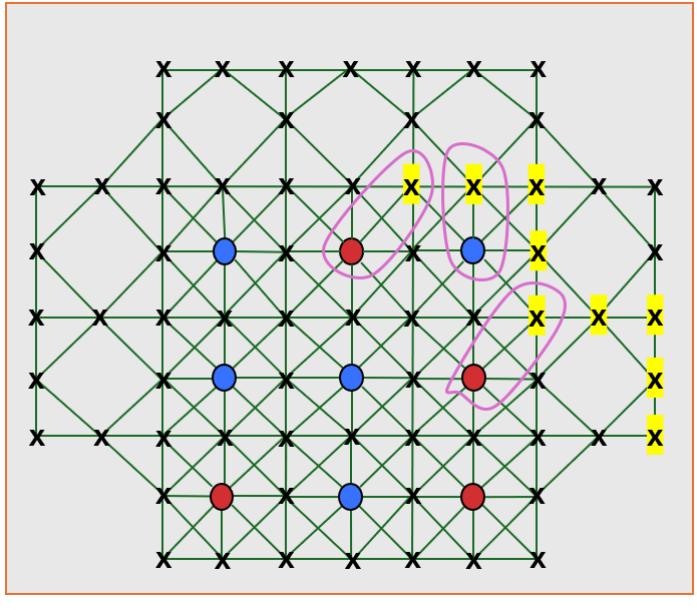
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple



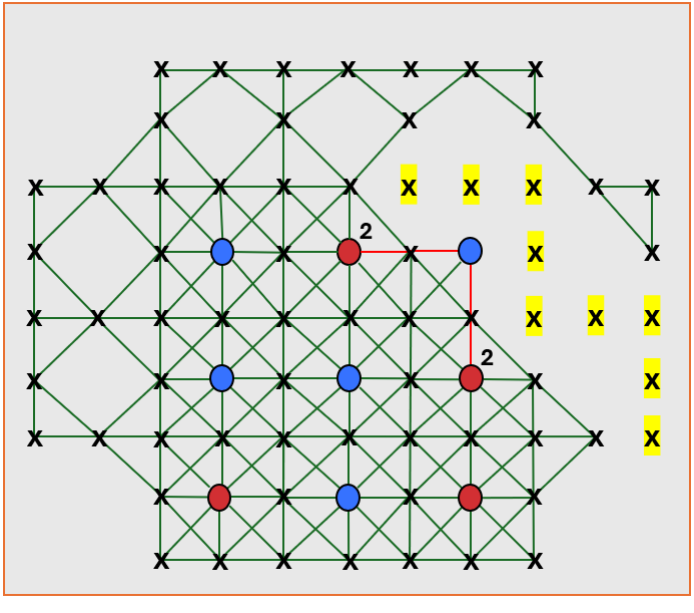
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple



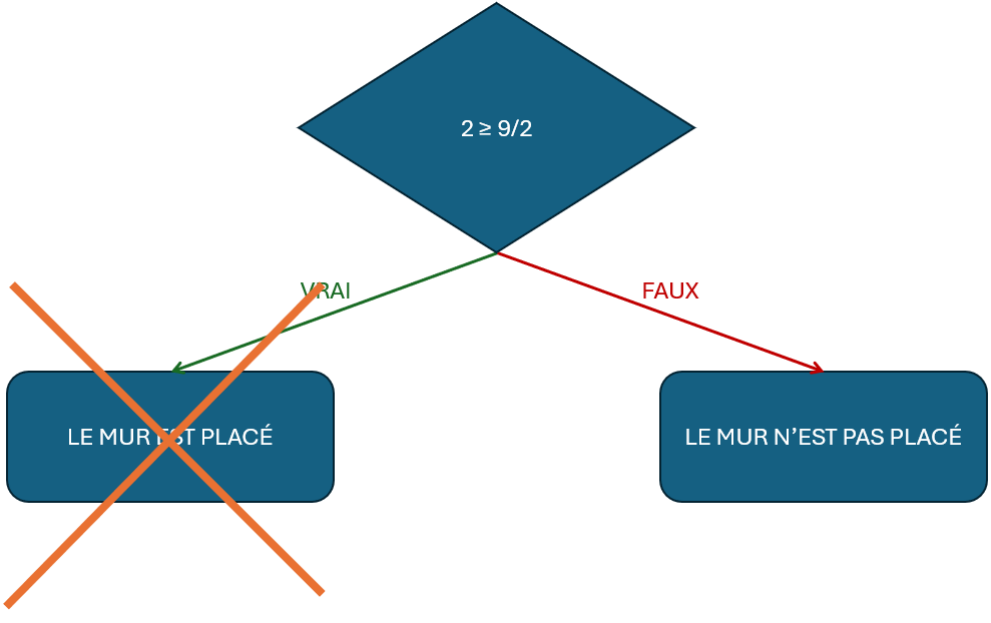
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple



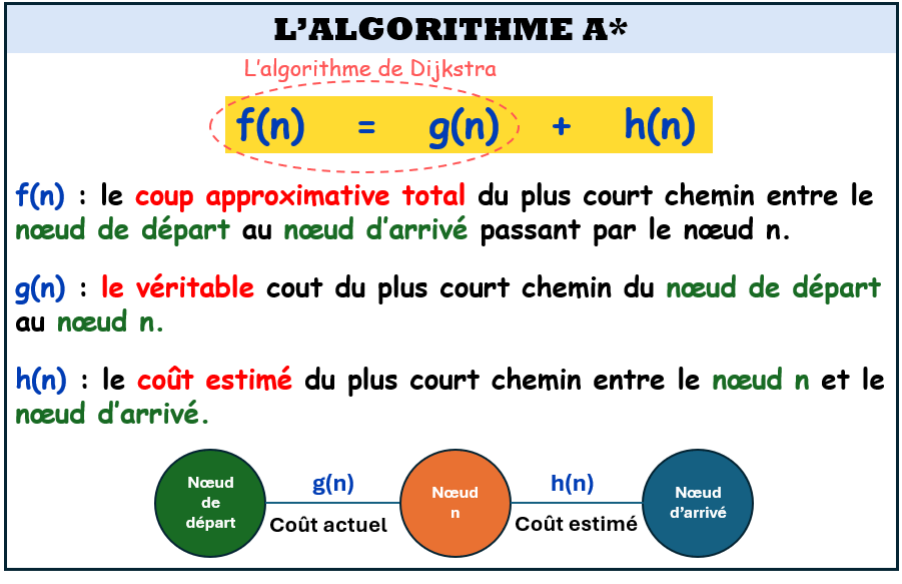
Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Explication de l'algorithme sur un exemple - exemple



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

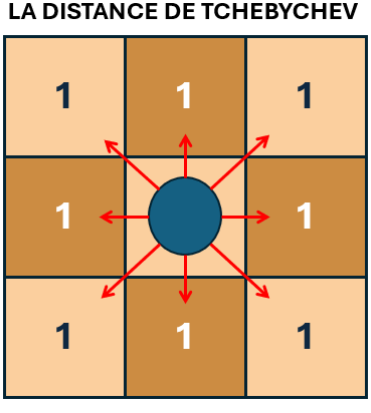
L'algorithme A*



Lee, Yumin. "A* Search Algorithm." *Medium*, 15 Jan. 2022, https://yuminlee2.medium.com/a-search-algorithm-42c1a13fcf9f.

Deuxième tentative d'optimisation : utilisation de l'algorithme A*

L'heuristique utilisé



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Résultats temporels



Deuxième tentative d'optimisation : utilisation de l'algorithme A*

Comparaison des résultats temporels



Annexe

La fonction : placer rempart

```
let placer_rempart (rempart : (int*int) array) ((x,y) : (int*int)) (f1,f2) (g : int array array) : bool =
  let n = ((Array.length g)-1)/2 in
  let p = ((Array.length g.(0))-1)/2 in
  let b = ref true in
  let k = ref 0 in
  (* for k = 0 to Array.length rempart -1 do *)
  while ((!k < Array.length rempart) && !b) do
    let (i,j) = rempart.(!k) in
    if (i+x < 0 || i+x > 2*n || j+y < 0 || j+y > 2*p || g.(i+x).(j+y) <> 1) then
      b := false (*le rempart depasserait de la grille*)
    else if ((i+x + j+y) mod 2 <> 0) then (*on est entre deux 'cases pions'*)
      begin
        (* print_string ("test - "); *)
        if ((j+y) mod 2 = 0 && ((i+x > 0 && g.(i-1+x).(j+y) <> 1) || (i+x < 2*n && g.(i+1+x).(j+y) <> 1))) then
          b := false; (*ces 'cases pions' sont à gauche et à droite*)
        if ((i+x) mod 2 = 0 && ((j+y > 0 && g.(i+x).(j-1+y) <> 1) || (j+y < 2*n && g.(i+x).(j+1+y) <> 1))) then
          b := false; (*ces 'cases pions' sont au dessus et en dessous*)
      end
    else
      if ((i+x) mod 2 = 0) then (*on est dans une case 'coin'*)
        begin (*pour placer une tour, il faut que toutes (max 4) les cases sur les côtés soient libres*)
          if (i+x > 0 && g.(i+x-1).(j+y) = 2 && (i+x-1,j+y) <> (f1,f2)) then
            b := false;
          if (i+x < 2*n && g.(i+x+1).(j+y) = 2 && (i+x+1,j+y) <> (f1,f2)) then
            b := false;
          if (j+y > 0 && g.(i+x).(j+y-1) = 2 && (i+x,j+y-1) <> (f1,f2)) then
            b := false;
          if (j+y < 2*p && g.(i+x).(j+y+1) = 2 && (i+x,j+y+1) <> (f1,f2)) then
            b := false;
        end
      else (*si on veut placer un rempart sur une une case pion*)
        b := false;
    k := !k +1
  done;
  if not(!b) then
    false
  else
    begin
      for k = 0 to Array.length rempart -1 do
        let (i,j) = rempart.(k) in
        g.(i+x).(j+y) <- 2 (*on place la case du rempart correspondante*)
      done;
      true
    end ;;
(*si la place est libre, la fonction place le rempart et renvoie true, elle renvoie false et ne fait rien sinon*)
```

Annexe

La fonction : est solution

```
let est_solution (g : int array array) : (int*int) list =
  let n = ((Array.length g)-1)/2 in
  let p = ((Array.length g.(0))-1)/2 in
  let cases_a_l_ext = Array.make_matrix n p false in
  let cases_a_utiliser_bord = ref [] in
  for i = 1 to n-2 do
    cases_a_utiliser_bord := (i,0) :: !cases_a_utiliser_bord;
    cases_a_utiliser_bord := (i,p-1) :: !cases_a_utiliser_bord;
  done;
  for j = 1 to p-2 do
    cases_a_utiliser_bord := (0,j) :: !cases_a_utiliser_bord;
    cases_a_utiliser_bord := (n-1,j) :: !cases_a_utiliser_bord;
  done;
  let pions_a_traiter = ref [] in
  for i = 0 to n-1 do
    for j = 0 to p-1 do
      if (g.(2*i+1).(2*j+1) = 3 || g.(2*i+1).(2*j+1) = 4) then
        pions_a_traiter := (i,j,g.(2*i+1).(2*j+1)) :: !pions_a_traiter;
    done;
  done; (*stocke un triplet (ligne,colonne,couleur) de chaque chevalier present sur la grille*)
  let mauvais_pions = ref [] in
```

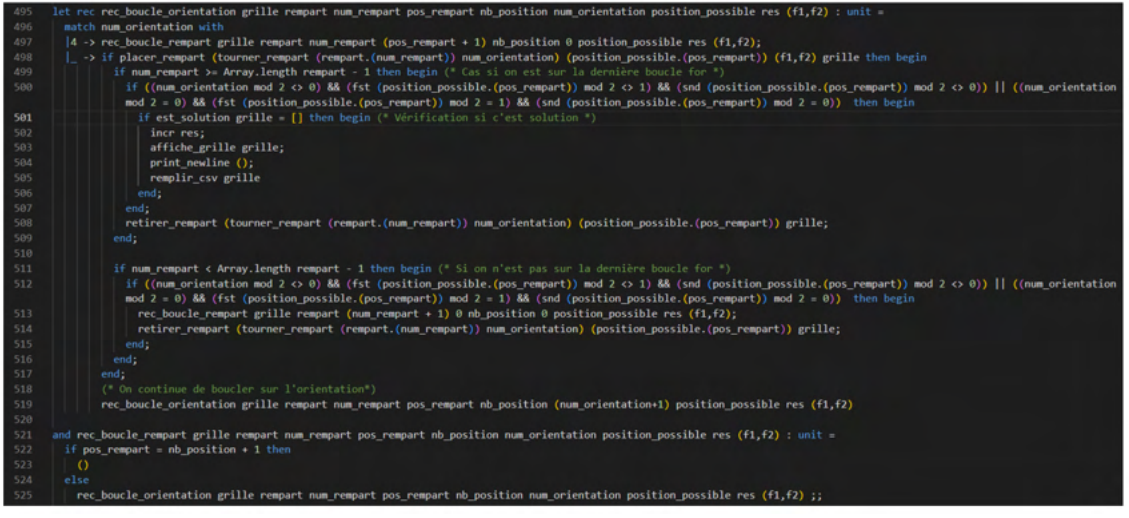
Annexe

La fonction : est solution suite

```
while (!cases_a_utiliser_bord <> []) do
  let (i,j) = List.hd (!cases_a_utiliser_bord) in
  if (cases_a_l_ext.(i).(j) = false) then
    begin
      match (i,j) with
      | (0,1) when (g.(1).(2) = 1 || g.(0).(3) = 1) ->
          cases_a_l_ext.(0).(1) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (0,b) when b = p-2 && (g.(1).(2*p-2) = 1 || g.(0).(2*p-3) = 1) ->
          cases_a_l_ext.(0).(p-2) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (0,b) when (g.(0).(2*b+1) = 1) ->
          cases_a_l_ext.(0).(b) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,1) when a = n-1 && (g.(2*n-1).(2) = 1 || g.(2*n).(3) = 1) ->
          cases_a_l_ext.(n-1).(1) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,b) when (a,b) = (n-1,p-2) && (g.(2*n-1).(2*p-2) = 1 || g.(2*n).(2*p-3) = 1) ->
          cases_a_l_ext.(n-1).(p-2) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,b) when a = n-1 && (g.(2*n).(2*b+1) = 1) ->
          cases_a_l_ext.(n-1).(b) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (1,0) when (g.(2).(1) = 1 || g.(3).(0) = 1) ->
          cases_a_l_ext.(1).(0) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,0) when a = n-2 && (g.(2*n-2).(1) = 1 || g.(2*n-3).(0) = 1) ->
          cases_a_l_ext.(n-2).(0) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,0) when (g.(2*a+1).(0) = 1) ->
          cases_a_l_ext.(a).(0) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (1,b) when b = p-1 && (g.(2).(2*p-1) = 1 || g.(3).(2*p) = 1) ->
          cases_a_l_ext.(1).(p-1) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,b) when (a,b) = (n-2,p-1) && (g.(2*n-2).(2*p-1) = 1 || g.(2*n-3).(2*p) = 1) ->
          cases_a_l_ext.(n-2).(p-1) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | (a,b) when b = p-1 && (g.(2*a+1).(2*p) = 1) ->
          cases_a_l_ext.(a).(p-1) <- true;
          remplir_voisins cases_a_l_ext (i,j) g
      | _ -> () (*print_string(" --- aucun cas"); print_newline()*) (*fin du 'if...then match with...'*)
    end;
  cases_a_utiliser_bord := List.tl (!cases_a_utiliser_bord)
done;
while (!pions_a_traiter <> []) do
  let (i,j,c) = List.hd (!pions_a_traiter) in
  if ((c = 3 && cases_a_l_ext.(i).(j) = false) || (c = 4 && cases_a_l_ext.(i).(j) = true)) then
    mauvais_pions := (i,j) :: !mauvais_pions;
  pions_a_traiter := List.tl (!pions_a_traiter)
done;
!mauvais_pions (*si la liste est vide, c'est que la grille est correcte, sinon non*)
```

Annexe

La fonction : backtracking



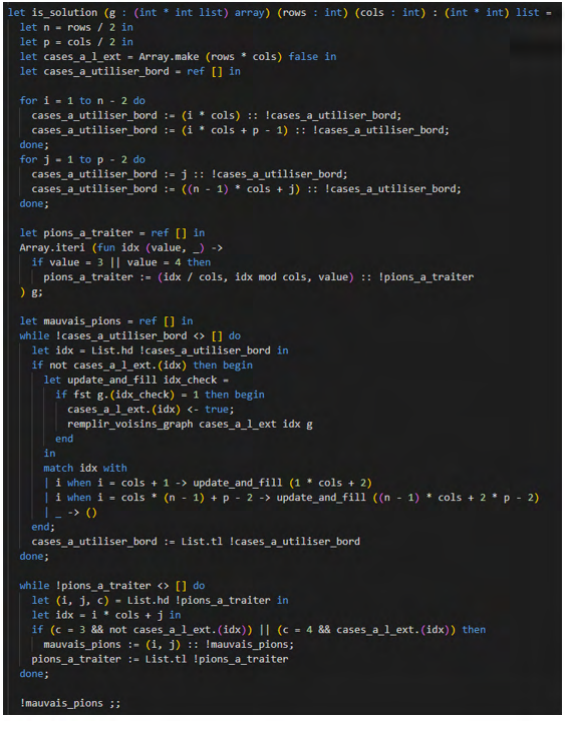
```
let rec rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) : unit =
  match num_orientation with
  |4 -> rec_boucle_rempart grille rempart num_rempart (pos_rempart + 1) nb_position 0 position_possible res (f1,f2);
  |_ -> if placer_rempart (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) (f1,f2) grille then begin
          if num_rempart >= Array.length rempart - 1 then begin (* Cas si on est sur la dernière boucle for *)
            if ((num_orientation mod 2 <> 0) && (fst (position_possible.(pos_rempart)) mod 2 <> 1) && (snd (position_possible.(pos_rempart)) mod 2 <> 0)) || ((num_orientation
            mod 2 = 0) && (fst (position_possible.(pos_rempart)) mod 2 = 1) && (snd (position_possible.(pos_rempart)) mod 2 = 0))  then begin
              if est_solution grille = [] then begin (* Vérification si c'est solution *)
                incr res;
                affiche_grille grille;
                print_newline ();
                remplir_csv grille
              end;
            end;
            retirer_rempart (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) grille;
          end;

          if num_rempart < Array.length rempart - 1 then begin (* Si on n'est pas sur la dernière boucle for *)
            if ((num_orientation mod 2 <> 0) && (fst (position_possible.(pos_rempart)) mod 2 <> 1) && (snd (position_possible.(pos_rempart)) mod 2 <> 0)) || ((num_orientation
            mod 2 = 0) && (fst (position_possible.(pos_rempart)) mod 2 = 1) && (snd (position_possible.(pos_rempart)) mod 2 = 0))  then begin
              rec_boucle_rempart grille rempart (num_rempart + 1) 0 nb_position 0 position_possible res (f1,f2);
              retirer_rempart (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) grille;
            end;
          end;
        end;
        (* On continue de boucler sur l'orientation*)
        rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position (num_orientation+1) position_possible res (f1,f2)

and rec_boucle_rempart grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) : unit =
  if pos_rempart = nb_position + 1 then
    ()
  else
    rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) ;;
```

Annexe

La fonction : est solution (pour un graph)



```
let is_solution (g : (int * int list) array) (rows : int) (cols : int) : (int * int) list =
  let n = rows / 2 in
  let p = cols / 2 in
  let cases_a_l_ext = Array.make (rows * cols) false in
  let cases_a_utiliser_bord = ref [] in

  for i = 1 to n - 2 do
    cases_a_utiliser_bord := (i * cols) :: !cases_a_utiliser_bord;
    cases_a_utiliser_bord := (i * cols + p - 1) :: !cases_a_utiliser_bord;
  done;
  for j = 1 to p - 2 do
    cases_a_utiliser_bord := j :: !cases_a_utiliser_bord;
    cases_a_utiliser_bord := ((n - 1) * cols + j) :: !cases_a_utiliser_bord;
  done;

  let pions_a_traiter = ref [] in
  Array.iteri (fun idx (value, _) ->
    if value = 3 || value = 4 then
      pions_a_traiter := (idx / cols, idx mod cols, value) :: !pions_a_traiter
  ) g;

  let mauvais_pions = ref [] in
  while !cases_a_utiliser_bord <> [] do
    let idx = List.hd !cases_a_utiliser_bord in
    if not cases_a_l_ext.(idx) then begin
      let update_and_fill idx_check =
        if fst g.(idx_check) = 1 then begin
          cases_a_l_ext.(idx) <- true;
          remplir_voisins_graph cases_a_l_ext idx g
        end
      in
      match idx with
      | i when i = cols + 1 -> update_and_fill (1 * cols + 2)
      | i when i = cols * (n - 1) + p - 2 -> update_and_fill ((n - 1) * cols + 2 * p - 2)
      | _ -> ()
    end;
    cases_a_utiliser_bord := List.tl !cases_a_utiliser_bord
  done;

  while !pions_a_traiter <> [] do
    let (i, j, c) = List.hd !pions_a_traiter in
    let idx = i * cols + j in
    if (c = 3 && not cases_a_l_ext.(idx)) || (c = 4 && cases_a_l_ext.(idx)) then
      mauvais_pions := (i, j) :: !mauvais_pions;
    pions_a_traiter := List.tl !pions_a_traiter
  done;

  !mauvais_pions ;;
```

Annexe

La fonction : branch and bound



```
let rec rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) adj_list remembered_points rows cols stop t :
unit =
  let t1 = Sys.time () in
  if t1 -. t > 5. then stop := true;
  match num_orientation,(!stop) with
  |_,true -> ();
  |4,_ -> rec_boucle_rempart grille rempart num_rempart (pos_rempart + 1) nb_position 0 position_possible res (f1,f2) adj_list remembered_points rows cols stop t;
  |_,_ -> if placer_rempart_graph (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) (f1,f2) adj_list rows cols && h adj_list
  remembered_points rows cols (float_of_int (Array.length grille / 3)) (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) then begin
            if num_rempart >= Array.length rempart - 1 then begin
              if ((num_orientation mod 2 <> 0) && (fst (position_possible.(pos_rempart)) mod 2 <> 1) && (snd (position_possible.(pos_rempart)) mod 2 <> 0)) ||
              ((num_orientation mod 2 = 0) && (fst (position_possible.(pos_rempart)) mod 2 = 1) && (snd (position_possible.(pos_rempart)) mod 2 = 0))  then begin
                let g = graph_to_grid adj_list rows cols in
                if est_solution g = [] then begin
                  incr res;
                  affiche_grille g;
                  stop := true;
                  print_newline ();
                  remplir_csv g
                end;
              end;
              remove_rempart_graph (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) adj_list rows cols;
            end;

          if num_rempart < Array.length rempart - 1 then begin
            if ((num_orientation mod 2 <> 0) && (fst (position_possible.(pos_rempart)) mod 2 <> 1) && (snd (position_possible.(pos_rempart)) mod 2 <> 0)) || ((num_orientation
            mod 2 = 0) && (fst (position_possible.(pos_rempart)) mod 2 = 1) && (snd (position_possible.(pos_rempart)) mod 2 = 0))  then begin
              rec_boucle_rempart grille rempart (num_rempart + 1) 0 nb_position 0 position_possible res (f1,f2) adj_list remembered_points rows cols stop t;
              remove_rempart_graph (tourner_rempart (rempart.(num_rempart)) num_orientation) (position_possible.(pos_rempart)) adj_list rows cols;
            end;
          end;
        end;
        rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position (num_orientation+1) position_possible res (f1,f2) adj_list remembered_points rows cols stop t

and rec_boucle_rempart grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) adj_list remembered_points rows cols stop t : unit =
  if pos_rempart = nb_position + 1 then
    ()
  else
    rec_boucle_orientation grille rempart num_rempart pos_rempart nb_position num_orientation position_possible res (f1,f2) adj_list remembered_points rows cols stop t;;
```